

Introduction to Service Portal

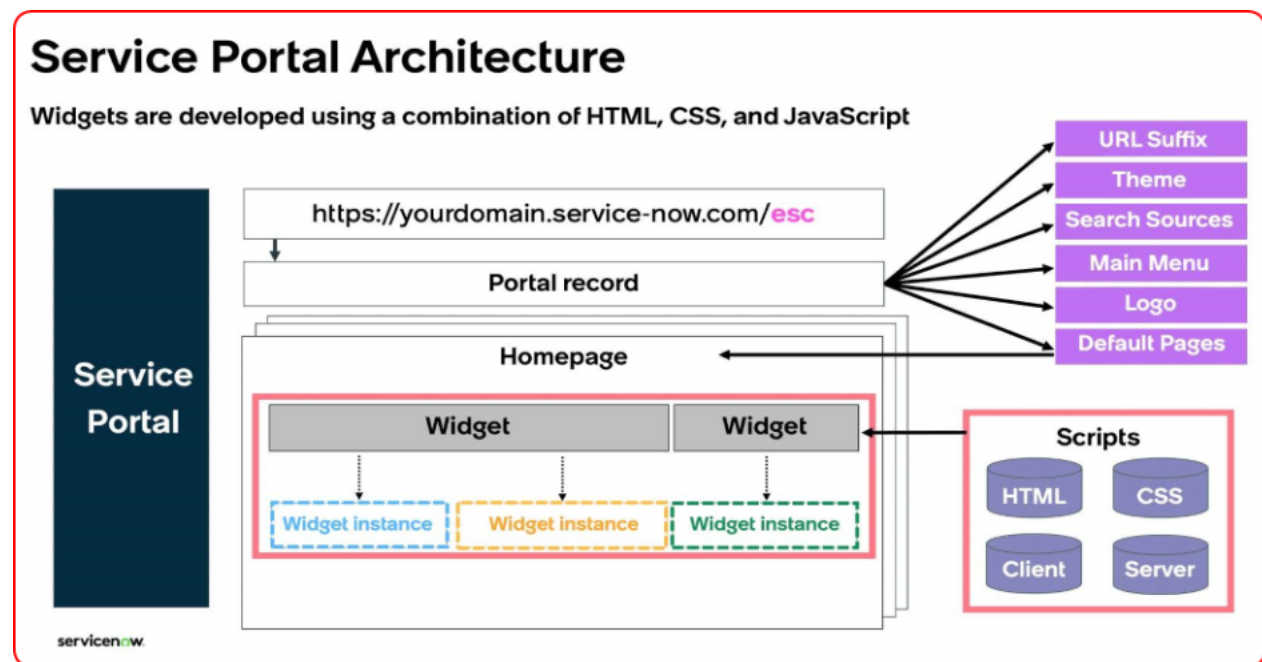
A well-designed Service Portal provides users with an engaging, intuitive, and efficient way to access applications, information, and services. It serves as a visual layer that transforms the platform's look and feel, ensuring consistent and seamless user experiences across devices. Service Portals allow users to find information, complete tasks, and submit service requests easily, helping organizations deliver modern, user-friendly digital experiences. Administrators and developers can customize portals to meet diverse organizational needs using either no-code, low-code, or pro-code configurations.

Service Portal architecture

A Service Portal is composed of interconnected elements that define how users access and interact with ServiceNow applications. The portal framework is built using HyperText Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript, integrating reusable components to deliver customized experiences. Key components of the portal architecture include:

- **Uniform Resource Locator (URL) suffix:** Defines the unique portal address.
- **Portal record:** Determines configuration such as theme, logo, and default pages.
- **Homepage:** Displays widgets that serve as containers for content and functionality.
- **Widgets and widget instances:** Provides structure and interactivity as reusable User Interface (UI) components.
- **Scripts (HTML, CSS, Client, Server):** Enable data retrieval, logic execution, and page styling.

The diagram below illustrates how Service Portal components interact within the overall architecture framework:



Portal widgets overview

Widgets are modular, reusable components that define how content and functionality appear in the Service Portal. They can display data, perform actions, or integrate with services.

Widgets can be:

- Added multiple times on a page with different configurations.
- Created or customized in the Widget Editor, using HTML, CSS, and JavaScript.
- Extended and reused across different pages to ensure consistency and reduce development effort.

Widget behavior is defined through:

- **HTML:** Renders user interface and binds data.
- **Server scripts:** Loads data and performs queries.
- **Client scripts:** Handles user interactions and data processing.